

SURREALISTIC TEQBALL

BabylonJS Web Game — Revised Development Plan v2

This is a revised edition addressing critical flaws identified in peer review of v1: missing testing strategy, dangerous parallel dependencies, undefined asset pipeline, unrealistic beginner timeline, absent CI/CD, and missing code quality tooling. A complete file map for GitHub initialization has been added as Section 16.

■ Genre Surrealistic Tegball Simulation	■ Platform Web (BabylonJS) Desktop + Mobile	■ Players 1v1 / 2v2 Local-screen first	■ Revised 12–14 weeks MVP With AI + team	■ v2 Adds Tests · CI/CD File map · Git LFS
--	---	---	--	--

Table of Contents

- A. What Changed from v1 — Full response to all 10 critique points
- 1. Phase 0 — Environment & Git Foundation — Week 1: repo, LFS, linting, CI/CD, hello world deploy
- 2. Phase 0.5 — Interfaces Contract — Week 1.5: shared TypeScript contracts before any feature work
- 3. Phase 1 — Table, Ball & Camera — Weeks 2–3: core physics, camera prototype, first deploy smoke test
- 4. Phase 2 — One Character — Weeks 4–5: input, state machine, simplified animation (no blending yet)
- 5. Phase 3 — Match Logic & Rules — Weeks 6–7: scoring, rule engine, HUD, integration sync point
- 6. Phase 4 — Superpowers & VFX — Weeks 8–9: abilities, particles, shaders
- 7. Phase 5 — Content & Polish — Weeks 10–12: 2+ arenas, 2+ characters, audio, post-process
- 8. Phase 6 — Buffer & Release — Weeks 13–14: bug fixing, performance, deployment
- 9. Testing Strategy — Unit, integration, physics reproducibility, manual QA
- 10. Dependency Map — What blocks what — per-member unblocking sequence
- 11. Asset Pipeline Protocol — Who creates, who optimizes, size limits, check-in rules
- 12. Code Quality Tooling — ESLint, Prettier, Husky, tsconfig strict mode
- 13. CI/CD Setup — GitHub Actions: lint, test, build, deploy on every PR
- 14. Multiplayer Clarification — Local 2v2 vs networked — honest scope definition
- 15. GitHub Repo & Branch Strategy — Branches, protection rules, PR process, Git LFS
- 16. Complete File Map — Every file in every folder — purpose mapped for GitHub init

A. What Changed from v1 — Response to Critique

[CRITICAL] Git workflow missing

Problem: v1 mentioned shared repository but had no branching strategy, no PR process, no Git LFS.

Fix in v2: Section 15 now has full branch strategy, PR rules, Git LFS setup. Phase 0 includes Git LFS as a Day 1 task. Section 16 maps every file for immediate repo init.

[CRITICAL] No testing strategy

Problem: v1 had zero mention of tests. Bugs compound fast in 3-person teams without them.

Fix in v2: Section 9 adds a full testing strategy: unit tests for RuleEngine and SuperpowerSystem, integration tests for physics, reproducible physics scenarios, manual QA checklist. Vitest is added to the stack. Phase 0 includes test setup.

[CRITICAL] Dangerous parallel dependencies

Problem: v1's parallel workstreams had members blocked waiting for each other's output mid-sprint.

Fix in v2: Section 10 maps every dependency explicitly. Section 2 (Phase 0.5) defines the interfaces.ts contract session that all three attend before any feature work begins. No member starts their phase until the shared contract is signed off.

[CRITICAL] Multiplayer labelled misleadingly

Problem: v1 said 'multiplayer local' without clarifying this meant one-keyboard 2v2, not networked.

Fix in v2: Section 14 now explicitly separates: Local 2v2 (one screen, shared keyboard/gamepads) which is in scope for MVP, versus Networked multiplayer (Colyseus.js) which is post-launch scope and clearly labelled as such.

[CRITICAL] Timeline unrealistic for beginners

Problem: v1 expected beginners to learn BabylonJS, Vite, Git, animation blending all in week 1.

Fix in v2: Timeline extended to 12–14 weeks for MVP. Phase 0 is now dedicated entirely to tooling. Animation blending removed from early phases — simplified animation first, blending in Phase 5. Buffer weeks 13–14 added for iteration.

[MODERATE] No code quality tooling

Problem: v1 had no ESLint, Prettier, or Husky. Code style diverges fast across 3 beginners.

Fix in v2: Section 12 covers full setup: ESLint with TypeScript rules, Prettier with shared config, Husky pre-commit hooks that run lint + typecheck before every commit. tsconfig strict mode.

[MODERATE] Asset pipeline undefined

Problem: v1 mentioned tools but not who does what, size limits, or when compression happens.

Fix in v2: Section 11 defines the complete asset pipeline protocol: creator, reviewer, size limits per asset type, Draco + KTX2 compression applied from Phase 1 onwards, not Phase 5.

[MODERATE] Deployment too late

Problem: v1 only deployed in the final week. No time to fix hosting issues.

Fix in v2: Phase 0 now includes a 'hello world' deploy to Vercel/Netlify on Day 3. Every phase ends with a deploy smoke test.

[MODERATE] Camera underestimated

Problem: v1 listed camera as a minor Phase 1 task. It is iterative and affects all gameplay testing.

Fix in v2: Camera is now prototyped in Phase 1 as a primary task with a fixed camera first, then follow camera, with a note it will be refined in every subsequent phase.

[MODERATE] No documentation or onboarding

Problem: v1 had no README, architecture decisions, or API docs. Code becomes cryptic quickly.

Fix in v2: Phase 0 includes writing the README with setup instructions, architecture overview, and branch rules. interfaces.ts acts as living API documentation. Each system's constructor is documented with JSDoc comments.

1. Phase 0 — Environment & Git Foundation

■ Week 1 (5 days)

■ Effort: Low–Medium

■ Focus: Foundation before any game code

Nothing else starts until this phase is complete. All three members do this together. The goal: every machine runs the project identically, the repo is protected, a hello-world 3D scene deploys to a public URL, and the linter blocks bad code before it merges.

Day	Task	Who	Done when
1	Create GitHub repo. Set branch protection on main and dev. Add all collaborators. Create .gitignore.	Dev A leads, all attend	All 3 can clone and see the repo
1	Install Git LFS. Run: <code>git lfs track '*.glb' '*.png' '*.ktx2' '*.ogg'</code> . Commit .gitattributes.	Dev A	.gitattributes committed, git lfs ls-files works
2	<code>npm create vite@latest teqball -- --template vanilla-ts</code> . Install BabylonJS, Havok. Verify blank canvas.	Dev A	<code>npm run dev</code> shows blank browser page
2	Install and configure ESLint + Prettier + Husky. Add .eslintrc, .prettierrc. Hook runs on commit.	Dev A	Committing a broken file triggers lint error and blocks commit
2	Write tsconfig.json with <code>strict:true</code> , <code>noImplicitAny:true</code> . Fix any errors the stricter config reveals.	Dev A	<code>npm run build</code> passes with zero TypeScript errors
3	Set up Vitest for unit testing. Write one smoke test: <code>1+1===2</code> . Confirm it passes via <code>npm run test</code> .	Dev A	<code>npm run test</code> outputs green
3	Set up GitHub Actions CI: on every PR to dev → run lint, typecheck, test, build. See Section 13.	Dev A	Push a test PR, watch Actions tab go green
3	Deploy hello-world scene to Vercel or Netlify. Get a public HTTPS URL. Share it with the team.	Dev A	All 3 members open the URL on their phones and see a 3D box
4	All members clone the repo. Each person runs <code>npm install</code> and <code>npm run dev</code> . Fix any machine-specific issues.	All	All 3 are running the project locally
4	Write README.md: setup steps, npm commands, branch rules, folder ownership, tech stack summary.	Dev A writes, all review	Someone unfamiliar can set up the project by following README only
5	Each member creates a test feature branch, makes a trivial change, opens a PR to dev, gets it reviewed and merged.	All 3 in turn	All 3 have completed the PR workflow at least once before real work starts

TypeScript config that must be set from day 1

```
// tsconfig.json
{ "compilerOptions": {
  "strict": true,
  "noImplicitAny": true,
  "strictNullChecks": true,
  "target": "ES2020",
  "module": "ESNext",
  "moduleResolution": "bundler",
  "lib": ["ES2020", "DOM"],
  "outDir": "./dist",
  "rootDir": "./src"
}
```

2. Phase 0.5 — Interfaces Contract (all 3 members together)

■ 2 days (end of Week 1)

■ Effort: Medium

■ Focus: Prevents all cross-member blocking

Before anyone writes a single game feature, all three members sit together (or on a call) and agree on the TypeScript interfaces that define how systems talk to each other. This file — `src/core/interfaces.ts` — is the contract. Dev B and Dev C can start building against these interfaces even while Dev A implements them. No interface changes without team discussion.

Minimum interfaces to define before feature work starts

Interface	Owned by	Used by	Key methods / properties
IEntity	Dev A	All	update(delta: number): void; dispose(): void
IBallSystem	Dev A	Dev B (VFX), Dev C (UI)	applyKickImpulse(dir, power); applyEffect(id); velocity: Vector3
ICharacter	Dev A	Dev B (animations), Dev C (UI)	state: CharacterState; stats: CharacterStats; activatePower(): void
IMatchManager	Dev A	Dev B (VFX), Dev C (UI)	onPoint: Observable; onMatchEnd: Observable; score: MatchScore
IInputManager	Dev A	Dev A (characters)	isActionPressed(action: GameAction): boolean
IVFXSystem	Dev B	Dev A (triggers)	playEffect(id: string, position: Vector3): void
IAudioSystem	Dev C	Dev A (triggers)	play(id: string, options?: SoundOptions): void
IArenaConfig	Dev C	Dev A (loads it)	id, skyboxPath, gravityModifier, ambientParticles, musicPath
CharacterConfig	Dev C	Dev A (loads it)	id, modelPath, stats, superpower, limitation, animations map

Example interfaces.ts starter

```
// src/core/interfaces.ts – DO NOT EDIT without team discussion
import { Observable } from '@babylonjs/core/Misc/observable';
import { Vector3 } from '@babylonjs/core/Maths/math.vector';

export type CharacterState = 'idle'|'running'|'jumping'|'kicking'|'using_power'|'stunned';
export type GameAction = 'move_left'|'move_right'|'jump'|'kick'|'use_power';

export interface IEntity { update(delta: number): void; dispose(): void; }

export interface IBallSystem extends IEntity {
  applyKickImpulse(direction: Vector3, power: number): void;
  applyEffect(effectId: string): void;
  readonly velocity: Vector3;
}

export interface IMatchManager {
  readonly onPoint: Observable<{ scorer: 'left'|'right' }>;
  readonly onMatchEnd: Observable<{ winner: 'left'|'right' }>;
  readonly score: { left: number; right: number };
}
```

3. Phase 1 — Table, Ball & Camera

■ Weeks 2–3

■ Effort: High

■ Focus: Core physics feel — most important phase

By the end of this phase: the ball drops, bounces satisfyingly off the table, and the camera follows it. The scene deploys. Physics tuning happens here, not later. Simplified animation (no blending yet) is acceptable at this stage.

Member	Task	Deliverable	Blocks who
Dev A	Load <code>teqball_table.glb</code> with <code>PhysicsAggregate</code> . Add net with collision. Tag meshes with metadata.	Table and net solid — ball doesn't fall through	Dev B (needs table mesh to test shaders)
Dev A	Create <code>BallSystem</code> class implementing <code>IBallSystem</code> . <code>SphereShape</code> , restitution 0.8, friction 0.4. <code>applyKickImpulse</code> working.	Ball falls, bounces, can be launched by code	All (ball is the game)
Dev A	Write <code>CollisionDetector</code> stub: log to console what hit what. No game logic yet.	Console shows: 'ball hit table:left at v=4.2'	Dev A Phase 2 (needs this for scoring)
Dev A	Write 3 unit tests in <code>tests/ball.test.ts</code> : spawn ball at height → assert it lands, apply impulse → assert velocity > 0, restitution sanity check.	npm run test passes all 3 ball tests	Nothing (tests are self-contained)
Dev B	Prototype two cameras: (A) fixed elevated side view, (B) <code>ArcRotateCamera</code> following ball. Test both. Pick A for now.	Camera renders the table from a playable angle	Dev C (UI needs a camera to test against)
Dev B	Basic toon shader on table and ball material. No outline yet. Just step-based diffuse.	Table and ball look cartoon, not grey/default	Dev A (needs a visual to show teammates)
Dev B	Set up <code>DefaultRenderingPipeline</code> with only bloom enabled. Threshold 0.8 (subtle).	Bloom visible on ball when it moves fast	Phase 4 (VFX layer exists now)
Dev C	Create first arena config JSON: <code>cloud_arena.json</code> . Fill all <code>IArenaConfig</code> fields with placeholder paths.	Valid JSON file that passes JSON schema validation	Dev A (needs config format to write loader)
Dev C	Build loading screen UI using <code>BabylonJS</code> GUI. Shows progress bar while assets load.	Loading screen visible, hides when assets ready	All (every phase needs this)
All	End of Phase 1: deploy to Vercel. All 3 open the URL. Ball bounces on table. Camera works.	Shared URL in team chat	Phase 2 cannot start until this smoke test passes

Physics tuning guide — do this in Phase 1, not later

Open a physics sandbox scene (separate from the game scene). Place the ball at $y=2$. Run the simulation. Adjust restitution (0.6–0.9) until the bounce height feels right. Adjust friction (0.2–0.6) until the ball rolls naturally. Write the final values as constants in `src/core/constants.ts` so they never get changed accidentally. Once locked, write a test that asserts the ball reaches a minimum bounce height.

4. Phase 2 — One Character (Simplified Animation)

■ Weeks 4–5

■ Effort: High

■ Focus: Input → movement → kick → ball reacts

Animation blending is deferred to Phase 5. In this phase, animations are switched directly (snap, not blend). This is intentional — blending is an advanced technique. Get movement and kicking working first. Polish later.

Member	Task	Deliverable	Blocks who
Dev A	InputManager: keyboard + gamepad support. Maps keys to GameAction enum. Supports 2 players on 1 keyboard.	WASD/Arrows move independently in test scene	All (input needed before anything interactive)
Dev A	Character class: load .glb, create PhysicsAggregate (CAPSULE shape), implement ICharacter, connect to InputManager.	Character mesh on screen, moves with keyboard	Dev B (needs character to attach animations)
Dev A	State machine: IDLE, RUNNING, JUMPING, KICKING, STUNNED only. No blending. Direct anim swap.	Character plays correct animation per state, no blending	Dev B (needs states to drive anims)
Dev A	Kick logic: on kick input, apply impulse to ball if within 1.5m range. Use kick anim frame callback.	Character kicks, ball flies in kick direction	Dev C (needs kick to test superpower hooks)
Dev A	Write unit tests for state machine: assert state transitions are valid (KICKING → IDLE only on anim end).	npm run test includes state machine tests	QA
Dev B	Retarget or download animations via Mixamo. Export each as separate .glb. Merge into character.glb in Blender.	character.glb with Idle, Run, Jump, Kick, Stun clips all named correctly	Dev A (needs this file to load animations)
Dev B	Character outline shader using NodeMaterial Editor. Export as JSON. Apply to character mesh.	Character has visible cartoon outline	Phase 4 (outline intensity increases on superpower)
Dev B	Dust particle system that triggers on ball bounce. Pre-build in BabylonJS Particle Editor, save as JSON.	Dust puff visible on ball impact with table	Phase 4 (VFX system exists)
Dev C	Character select screen: two placeholder cards, clicking one sets active character config. No 3D preview yet.	Can select player 1 and player 2 character before match	Dev A (needs selected config to load character)
Dev C	Placeholder HUD: score display (0 : 0) and a serve indicator. No animations yet.	HUD visible during gameplay	Phase 3 (MatchManager will update these)

Why animation blending is deferred

Animation blending (weight-based cross-fade between clips) requires all animations to share the same bind pose and be rigged consistently. Getting this right takes iteration. In Phase 2, snap-switching animations is sufficient to validate gameplay. Blending is added in Phase 5 as a polish pass, when the character pipeline is stable. This unblocks Phase 2 Dev B by ~1 week.

5. Phase 3 — Match Logic & Rules

■ Weeks 6–7

■ Effort: High

■ Focus: MVP integration sync point

Phase 3 ends with the first fully playable match. This is the integration sync point — all three members' work from Phases 1–2 is assembled and tested together for the first time. Block 2 days at the end of this phase for integration debugging.

Member	Task	Deliverable	Blocks who
Dev A	MatchManager implementing IMatchManager: score tracking, serve rotation, set count, fault detection.	Points score correctly. Sets track. Match ends.	Dev C (UI subscribes to onPoint)
Dev A	RuleEngine: validate each touch — max 3 contacts, ball must cross net, no hand contacts. Returns fault type.	Faults are detected and logged. Rally resets on fault.	Dev C (fault UI needs this)
Dev A	EventBus: central singleton with emit/on. All systems communicate through this. No direct cross-system calls.	MatchManager, AudioSystem, VFXSystem all talk via EventBus	All (required for loose coupling)
Dev A	Write integration test: spawn match → simulate ball crossing net → assert point scored for correct side.	Integration test passes in CI	QA
Dev B	Score-reactive VFX: listen to EventBus 'point:scored' → play celebration particle burst at scorer's side.	Particles fire when point scored	Dev C (audio needs VFX to know timing)
Dev B	Arena atmosphere: load cloud_arena.json config, apply skybox, fog, ambient particles, gravity modifier.	Match takes place inside Cloud Arena with full atmosphere	Phase 5 (other arenas same pattern)
Dev C	HUD wired to MatchManager: subscribe to onPoint, update score text blocks. Serve indicator follows serve state.	HUD updates in real time during a match	QA
Dev C	Point announcement overlay: on onPoint → animated text 'POINT LEFT / POINT RIGHT', shows 2s, fades.	Overlay appears on each point	UX polish
Dev C	Post-match screen: winner display, stats (points per set), Play Again / Menu buttons.	Can complete a match and return to menu	Release
All	Integration day: all 3 members play a full match. Log bugs to GitHub Issues. Fix blockers before Phase 4.	Playable 1v1 match on shared URL	■ MILESTONE — MVP core loop

6. Phase 4 — Superpowers & VFX

■ Weeks 8–9

■ Effort: High

■ Focus: Surrealistic identity

Member	Task	Deliverable
Dev A	SuperpowerSystem: IAbility interface (activate/tick/deactivate), cooldown tracking, duration timer, EventBus integration.	Powers have cooldowns, can be activated, emit events
Dev A	Implement 4 abilities: GravityFlip, SpeedBurst, BallFreeze, MegaBounce. Each modifies BallSystem or Character.	All 4 abilities work with correct physics effects
Dev A	Limitation system: read 'limitation' from CharacterConfig, apply passive constraint on that character only.	One character cannot sprint, another cannot double-jump
Dev B	One unique VFX set per ability: particles + shader effect. Gravity flip = floating particle ring. Speed burst = trail streak.	Each power has a distinct visual signature
Dev B	Post-processing intensification: on power activation, momentarily increase bloom + chromatic aberration for 0.5s.	Screen flashes on superpower activation
Dev B	Animation blending: now that character pipeline is stable, replace snap-switch with weight-based blend over 0.2s.	Character transitions smoothly between states
Dev C	Cooldown ring UI: disc mesh below player feet, angle fills as cooldown recharges. Glows when ready.	Visual cooldown indicator visible in-game
Dev C	Superpower audio: activation sound, loop during active phase, deactivation sound. One per ability.	Each power has distinct sound identity
Dev C	Second character: complete config JSON, .glb sourced from Meshy or Mixamo, different stats and superpower.	Two fully playable characters with different abilities

7. Phase 5 — Content & Polish

■ Weeks 10–12

■ Effort: Medium–High

■ Focus: Diversity, replayability, visual quality

Member	Task	Deliverable
Dev A	ArenaVariationSystem: randomly pick arena, time of day, color LUT, gravity modifier before each match.	Every match feels different in same arena
Dev A	2v2 doubles: route 4 player inputs (2 keyboards or keyboard + 2 gamepads). Widen camera for full court.	2v2 match playable on one screen
Dev A	Basic AI opponent: ball position → move toward it → kick when in range. No pathfinding needed.	Single player can play against AI
Dev B	2 additional arenas: Underground Rave and Space Station. Full visual pass including atmosphere.	3 distinct arenas selectable from menu
Dev B	Performance audit: BabylonJS Inspector → check draw calls, texture memory, particle counts. Fix any >10ms frame budget item.	Game runs at 60fps on mid-range hardware
Dev B	LUT color grading: one per arena. Warm for Ruins, cool for Space, neon for Rave.	Each arena has a distinct color mood
Dev C	Full audio pass: all 6 SFX (bounce, kick, score, fault, power-activate, power-deactivate). Music per arena.	Game has complete audio, no silent moments
Dev C	Arena select screen: show 3 arenas with preview image and name. Selectable before match.	Players can choose arena before starting
Dev C	Character select with 3D preview: render character in separate scene, show in card. Stats bars.	Full character select flow
Dev C	Third character: sourced and integrated with unique superpower and limitation.	3 playable characters total

8. Phase 6 — Buffer & Release

■ Weeks 13–14

■ Effort: Medium

■ Focus: Stability, QA, deployment

These two weeks are deliberately unplanned feature-wise. They exist for what always happens: bugs found by playing, performance fixes, UI rough edges, and deployment surprises. Do not add new features here.

Task	Owner	Done when
Full manual QA session: 2 hours of continuous play, log every bug to GitHub Issues.	All 3 members	Issue list is created and prioritized
Fix all P1 bugs (gameplay-breaking). P2 bugs (annoying but not breaking) if time allows.	Assigned per bug area ownership	No P1 bugs open
Performance profile: target 60fps on a 4-year-old laptop. Use BabylonJS Inspector.	Dev B	60fps sustained on test device
Optimize worst offenders: reduce particles, lower texture resolution, use mesh instancing.	Dev B	Draw calls < 150 per frame
Verify all assets are Git LFS tracked. Repository size < 500MB.	Dev A	git lfs ls-files shows all binary assets
Final deploy to production URL (Vercel / Netlify / GitHub Pages).	Dev A	Stable HTTPS URL shared publicly
Write short play guide in README: controls, how superpowers work, arena list.	Dev C	Someone new can understand the game without being told
Tag the release on GitHub: git tag v1.0.0 → git push origin v1.0.0.	Dev A	Release visible in GitHub Releases tab

9. Testing Strategy

■ Ongoing from Phase 0

■ Effort: Medium

■ Focus: Prevents bug compounding in teams

Three-person teams without tests will spend 30% of their time re-fixing bugs that were already fixed. Tests don't need to be exhaustive — targeted tests on the most fragile systems are enough. Use Vitest (fast, TypeScript-native, no config needed).

Test type	Tool	What to test	When written	Files
Unit — RuleEngine	Vitest	Ball lands left side: point for right. Double touch: fault. Net touch: fault. Each rule as its own test.	Phase 3, before MatchManager	tests/ruleEngine.test.ts
Unit — State machine	Vitest	KICKING cannot transition to JUMPING. STUNNED ignores all inputs. Valid transitions only.	Phase 2, with state machine	tests/characterState.test.ts
Unit — Superpower cooldown	Vitest	Power not activatable at t=0 (cooldown). Activatable at t>cooldown. Duration expires correctly.	Phase 4, with SuperpowerSystem	tests/superpower.test.ts
Unit — EventBus	Vitest	emit('event', data) → registered listener receives data. Unsubscribe works. No memory leak.	Phase 2, with EventBus	tests/eventBus.test.ts
Integration — Match flow	Vitest + mock scene	Simulate: serve → rally → ball crosses net → point scored → correct side wins set.	Phase 3, integration day	tests/matchFlow.test.ts
Physics reproducibility	Vitest + BabylonJS headless	Same initial conditions → same ball trajectory every run. Catches physics regression.	Phase 1, after ball physics locked	tests/physicsSanity.test.ts
Manual QA checklist	Google Sheet	Ball doesn't tunnel through table. Character doesn't clip into net. Powers don't stack. Score resets between sets.	Phase 3 + Phase 6	docs/qa-checklist.md

Example unit test — RuleEngine

```
// tests/ruleEngine.test.ts
import { describe, it, expect } from 'vitest';
import { RuleEngine } from '../src/gameplay/RuleEngine';

describe('RuleEngine', () => {
  it('awards point to right side when ball lands on left side', () => {
    const result = RuleEngine.evaluate({ ballLandedSide: 'left', lastTouchedBy: 'left' });
    expect(result.type).toBe('point');
    expect(result.scorer).toBe('right');
  });
  it('calls double touch fault when same player touches twice', () => {
    const result = RuleEngine.evaluate({ touchCount: 2, lastTouchedBy: 'left', touchedBy: 'left' });
    expect(result.type).toBe('fault');
    expect(result.faultType).toBe('double_touch');
  });
});
```

10. Dependency Map — What Blocks What

This map shows which deliverable must exist before another can start. Use it at the beginning of each phase to confirm no member is blocked. If a dependency is late, the blocked member works on non-dependent tasks from the next phase.

Deliverable	Depends on	Unblocked by	Risk if late
Character loads in scene (Dev A Ph2)	character.glb with rig (Dev B Ph2)	Dev B delivers .glb by end of Week 3	Dev A cannot test movement without mesh
Animation system (Dev B Ph2)	State machine defined (Dev A Ph2)	Dev A delivers CharacterState enum in interfaces.ts (Phase 0.5)	Dev B animates wrong states
HUD wired to score (Dev C Ph3)	IMatchManager interface + onPoint Observable (Dev A Ph3)	Defined in interfaces.ts (Phase 0.5). Dev A implements by mid-Phase 3	HUD has no data to show
VFX on point scored (Dev B Ph3)	EventBus with 'point:scored' event (Dev A Ph3)	EventBus created in Phase 3 Week 1	VFX has nothing to listen to
Superpower VFX (Dev B Ph4)	IBallSystem.applyEffect() implemented (Dev A Ph4)	Dev A implements SuperpowerSystem Week 8	Dev B VFX has no trigger hook
Arena config loaded (Dev A Ph3)	IArenaConfig JSON written (Dev C Ph1)	Dev C delivers cloud_arena.json by end of Phase 1	Dev A cannot test arena loading
Character select functional (Dev C Ph2)	ICharacter + CharacterConfig type (Dev A Ph0.5)	Defined in interfaces.ts. No runtime dependency.	Character select has wrong config shape
2v2 input (Dev A Ph5)	InputManager supports 2 players (Dev A Ph2)	Dev A builds this in Phase 2	Self-dependency — sequential, not parallel

Unblocking protocol

If a member is blocked because a dependency from another member is not ready: (1) Post in team chat immediately — do not wait. (2) The blocked member pulls forward a non-dependent task from the next phase. (3) The blocking member prioritizes their deliverable above new features. (4) At the Monday sync, review if the dependency caused a phase slip and adjust the timeline. Never work around a missing interface by duplicating its logic — always wait and use the shared contract.

11. Asset Pipeline Protocol

■ Defined in Phase 0, applied from Phase 1

■ Effort: Low–Medium

■ Focus: Prevents repo bloat & style inconsistency

Size limits — enforced from Phase 1

Asset type	Max file size	Format	Compression required
Character .glb	8 MB	.glb	Draco mesh + KTX2 textures via gltf-transform
Arena .glb	20 MB	.glb	Draco mesh + KTX2 textures via gltf-transform
Texture (diffuse)	2 MB	.ktx2	Compress from source PNG using toktx CLI
HDR environment	8 MB	.env	Convert from .hdr using BabylonJS env tool
Music loop	4 MB	.ogg	Encode at 128kbps mono or 192kbps stereo
SFX	200 KB	.ogg	Encode at 96kbps mono
Particle config	50 KB	.json	No binary compression needed
Shader (NME)	100 KB	.json	No binary compression needed

Who does what in the asset pipeline

Step	Who	Tool	Output
1. Concept generation	Dev C	Midjourney / DALL-E	Reference image for 3D modeler
2. 3D model generation/modeling	Dev C	Meshy.ai / Blender	Raw .glb or .fbx (unoptimized)
3. Rigging (characters only)	Dev C	Mixamo (humanoids) / Blender	Rigged mesh, ready for animation
4. Animation creation	Dev B	Mixamo / Cascadeur / Blender	Animated clips, export each as .glb
5. Animation merge	Dev B	Blender (NLA editor)	Single character.glb with all clips
6. Optimization + compression	Dev B	gltf-transform CLI	character.min.glb (Draco + KTX2)
7. Preview validation	Dev B	BabylonJS Sandbox	Drag-drop .glb, confirm it loads
8. Check-in via PR	Dev B	Git LFS	Asset in assets/ folder, PR reviewed by Dev A
9. Integration in scene	Dev A	SceneLoader.ImportMesh Async	Asset loads in game scene

gltf-transform optimization command

```
# Install once
npm install -g @gltf-transform/cli

# Optimize a character (run this on every .glb before committing)
npx @gltf-transform/cli optimize raw/flamingo.glb assets/models/flamingo.glb \
--texture-compress ktx2 \
--draco-method edgebreaker

# Verify output size is under 8MB
ls -lh assets/models/flamingo.glb
```

12. Code Quality Tooling

■ Phase 0 setup, ~3 hours total

■ Effort: Low

■ Focus: Pays back 10x by Phase 3

ESLint + Prettier setup

```
# Install all code quality tools
npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin
npm install -D prettier eslint-config-prettier eslint-plugin-prettier
npm install -D husky lint-staged

# Initialize Husky
npx husky init
echo 'npx lint-staged' > .husky/pre-commit
```

Configuration files

```
// .eslintrc.json
{ "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint"],
  "extends": ["eslint:recommended", "plugin:@typescript-eslint/recommended", "prettier"],
  "rules": { "no-console": "warn", "@typescript-eslint/no-explicit-any": "error" }
}

// .prettierrc
{ "semi": true, "singleQuote": true, "tabWidth": 2, "printWidth": 100 }

// package.json – lint-staged section
"lint-staged": {
  "*.ts": ["eslint --fix", "prettier --write"]
}
```

What each tool catches

Tool	Catches before runtime	When it runs
TypeScript strict mode	Wrong types, null access, implicit any — class of bugs beginners make constantly	On every file save in VS Code + on build
ESLint	Unused variables, console.log left in, accessing undefined properties, inconsistent style	On pre-commit hook + in CI
Prettier	Inconsistent indentation, quote styles, trailing commas — keeps 3 people's code looking the same	On pre-commit hook automatically
Husky pre-commit hook	Blocks the commit if any lint or typecheck fails — code cannot enter the repo broken	On every git commit
Vitest in CI	Regressions — if a new commit breaks an existing feature, the test catches it before it merges	On every PR to dev or main

13. CI/CD Setup — GitHub Actions

■ Phase 0 Day 3, ~2 hours

■ Effort: Low

■ Focus: Automated quality gate on every PR

Every Pull Request to dev or main automatically runs: lint check, TypeScript build, unit tests. If any step fails, the PR cannot be merged. This prevents broken code from entering the shared codebase. Deployment to Vercel happens automatically when a PR merges to main.

GitHub Actions workflow file

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
    branches: [main, dev]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint # ESLint must pass
      - run: npm run typecheck # tsc --noEmit must pass
      - run: npm run test # Vitest must pass
      - run: npm run build # Vite build must succeed
```

Add these scripts to package.json

```
"scripts": {
  "dev": "vite",
  "build": "tsc && vite build",
  "preview": "vite preview",
  "lint": "eslint . --ext .ts",
  "typecheck": "tsc --noEmit",
  "test": "vitest run",
  "test:watch": "vitest"
}
```

Vercel auto-deploy setup (10 minutes)

1. Go to vercel.com → Import Git Repository → select your GitHub repo
2. Framework: Vite. Build command: `npm run build`. Output dir: `dist`.
3. Enable: Deploy on push to main.
4. Every merge to main → Vercel rebuilds → new version live within 60 seconds.
5. Every PR also gets a preview URL — share it with teammates to review before merging.

14. Multiplayer Clarification — Honest Scope Definition

v1 used the term 'multiplayer' without being clear about what it meant. There are two completely different things called 'multiplayer' and confusing them leads to massive scope overrun. This section defines both precisely.

Type	What it means	Complexity	In MVP scope?	How to implement
Local 2v2 (same screen)	2 or 4 players on one device. Each player uses different keys or a separate gamepad. No network code. No server.	Low	■ YES — Phase 2	InputManager handles 2 keyboard layouts and up to 4 gamepads via BabylonJS DeviceSourceManager
Networked (online)	Players on different devices, different networks. Requires a server to sync game state 60x/sec. Latency compensation required.	Very High	■ NO — post-launch	Colyseus.js server + client-side prediction. Entire separate architecture. Minimum 4–6 additional weeks.

Decision: Local 2v2 is in scope for the MVP. It requires only that the InputManager routes inputs to the correct character — no server, no latency, no sync. Networked multiplayer is a post-launch feature that requires its own sprint. Do not attempt to add it during the 14-week MVP timeline.

Local 2v2 input implementation

```
// Player 1: WASD + F to kick + G for power
// Player 2: Arrow keys + L to kick + ; for power
// OR: Player 1 on Gamepad 0, Player 2 on Gamepad 1
const P1_BINDINGS: InputBindings = {
  move_left: { keyboard: 'KeyA', gamepad: 'LeftStickLeft' },
  kick: { keyboard: 'KeyF', gamepad: 'ButtonX' },
  use_power: { keyboard: 'KeyG', gamepad: 'ButtonY' },
};
// InputManager.getActionState(playerId: 0|1, action: GameAction): boolean
```

15. GitHub Repo & Branch Strategy

■ Phase 0 Day 1

■ Effort: Low

■ Focus: Foundation of team workflow

Branch structure

Branch	Purpose	Who pushes	Merge rule
main	Production. Always deployable. Every merge triggers Vercel deploy.	Nobody directly	PR from dev only. Requires 2 approvals.
dev	Integration. All features land here first. Tested before main merge.	Nobody directly	PR from feature/* or fix/*. Requires 1 approval + CI green.
feature/[name]	One branch per task (e.g. feature/ball-physics). Created from dev.	The owner of the task	PR to dev. Requires 1 approval + CI green.
fix/[name]	Bug fixes (e.g. fix/ball-tunneling). Created from dev.	The bug owner	PR to dev. Requires 1 approval + CI green.
assets/[name]	Large binary asset additions. Created from dev. Skips lint CI (binaries).	Dev B or Dev C	PR to dev. Requires Dev A review of asset size.

Day 1 setup commands

```
# Clone and set up
git clone https://github.com/yourorg/teqball-game
cd teqball-game
npm install

# Set up Git LFS
git lfs install
git lfs track '*.glb' '*.png' '*.jpg' '*.ktx2' '*.ogg' '*.mp3' '*.env' '*.hdr'
git add .gitattributes
git commit -m 'chore: set up Git LFS tracking'

# Create dev branch
git checkout -b dev
git push -u origin dev

# From now on: always work on feature branches
git checkout dev && git pull origin dev
git checkout -b feature/your-task-name
```

Commit message convention

Use a prefix so the git log is readable: feat: (new feature), fix: (bug fix), chore: (tooling/config), assets: (adding/updating 3D assets), test: (test files only), docs: (README/docs). Example: feat: implement gravity flip superpower with ball physics override

16. Complete File Map for GitHub Initialization

Every file listed below should be created at project initialization. Stub files (empty class with correct exports) are sufficient — the goal is that the repo structure is complete from day 1 so no member creates files in unexpected locations. Files marked STUB can be empty classes. Files marked REAL must be complete before Phase 1 begins.

Root (project root)

File	Status	Owner	Purpose
<code>index.html</code>	REAL	Dev A	Entry point. Loads the compiled JS bundle and mounts the element.
<code>vite.config.ts</code>	REAL	Dev A	Vite bundler configuration. Sets base URL, aliases, and BabylonJS optimizations.
<code>tsconfig.json</code>	REAL	Dev A	TypeScript compiler options. Strict mode enabled. See Section 12.
<code>.eslintrc.json</code>	REAL	Dev A	ESLint rules for TypeScript. Prevents common runtime errors and style drift.
<code>.prettierrc</code>	REAL	Dev A	Prettier formatting rules shared by all 3 members.
<code>.gitignore</code>	REAL	Dev A	Excludes <code>node_modules/</code> , <code>dist/</code> , <code>.env</code> , OS files from Git.
<code>.gitattributes</code>	REAL	Dev A	Git LFS tracking rules for <code>.glb</code> , <code>.png</code> , <code>.ogg</code> , <code>.ktx2</code> , <code>.env</code> , <code>.hdr</code> files.
<code>package.json</code>	REAL	Dev A	Dependencies, scripts (dev, build, lint, test, typecheck). Source of truth for versions.
<code>README.md</code>	REAL	Dev A	Setup instructions, npm commands, branch rules, folder ownership, controls guide.

.github/workflows/

File	Status	Owner	Purpose
<code>ci.yml</code>	REAL	Dev A	GitHub Actions: runs lint, typecheck, test, build on every PR to dev or main.
<code>PULL_REQUEST_TEMPLATE.md</code>	REAL	Dev A	PR checklist: describe change, confirm tests pass, note which system is affected.

src/core/

File	Status	Owner	Purpose
<code>Engine.ts</code>	REAL	Dev A	Creates BabylonJS Engine on the canvas. Singleton. Starts the render loop. Exports engine instance.
<code>SceneManager.ts</code>	STUB	Dev A	Manages scene lifecycle: create, switch between menu/game scenes, dispose.
<code>AssetManager.ts</code>	STUB	Dev A	Wraps BabylonJS AssetsManager. Queues all <code>.glb</code> , texture, audio loads. Fires <code>onReady</code> when done.
<code>EventBus.ts</code>	REAL	Dev A	Central pub/sub. Methods: <code>on(event, cb)</code> , <code>emit(event, data)</code> , <code>off(event, cb)</code> . No cross-system direct calls.
<code>interfaces.ts</code>	REAL	Dev A	All shared TypeScript interfaces: <code>IEntity</code> , <code>IBallSystem</code> , <code>ICharacter</code> , <code>IMatchManager</code> , etc. See Section 2.
<code>constants.ts</code>	REAL	Dev A	Game constants: <code>BALL_RESTITUTION</code> , <code>BALL_FRICTION</code> , <code>TABLE_WIDTH</code> , <code>NET_HEIGHT</code> , gravity. Never magic numbers.
<code>GameLoop.ts</code>	STUB	Dev A	Registers all <code>IEntity</code> instances. Calls <code>update(delta)</code> on each in correct order each frame.

src/systems/

File	Status	Owner	Purpose
InputManager.ts	STUB	Dev A	Reads keyboard + gamepad via DeviceSourceManager. Maps inputs to GameAction per player ID.
PhysicsWorld.ts	STUB	Dev A	Initialises HavokPlugin. Sets gravity. Exposes helper to create PhysicsAggregates with correct groups.
CollisionDetector.ts	STUB	Dev A	Listens to Havok collision observables. Identifies mesh pairs via metadata. Emits typed EventBus events.
CameraManager.ts	STUB	Dev B	Creates and manages ArcRotateCamera. Exposes setTarget(), zoomOut() for doubles. Tweens on events.
RenderPipeline.ts	STUB	Dev B	Creates DefaultRenderingPipeline. Exposes methods to intensify/reduce bloom and chromatic aberration.

src/entities/

File	Status	Owner	Purpose
Character.ts	STUB	Dev A	Implements ICharacter. Wraps mesh + PhysicsAggregate + AnimationSystem. Drives state machine.
Ball.ts	STUB	Dev A	Implements IBallSystem. SphereShape physics. applyKickImpulse, applyEffect. TrailMesh attached.
Arena.ts	STUB	Dev A	Loads arena .glb. Applies IArenaConfig: skybox, fog, gravity modifier, ambient particles.
TeqballTable.ts	STUB	Dev A	Loads teqball_table.glb. Creates PhysicsAggregates for surface and net. Tags meshes with metadata.

src/gameplay/

File	Status	Owner	Purpose
MatchManager.ts	STUB	Dev A	Implements IMatchManager. Score, sets, serve rotation, match point. Emits via Observables.
RuleEngine.ts	REAL	Dev A	Pure functions. evaluate(RallyEvent) → RuleResult. No BabylonJS dependency. Fully unit-testable.
SuperpowerSystem.ts	STUB	Dev A	Manages active IAbility instances. Cooldown timers. Calls activate/tick/deactivate each frame.
abilities/GravityFlip.ts	STUB	Dev A	IAbility: reverses ball gravity scale for 3s. Emits 'ability:activated' on EventBus.
abilities/SpeedBurst.ts	STUB	Dev A	IAbility: multiplies character move speed by 3 for 2s. Leaves speed trail particle trigger.
abilities/BallFreeze.ts	STUB	Dev A	IAbility: zeroes ball velocity and locks it for 1s. Emits 'ball:frozen' event.
abilities/MegaBounce.ts	STUB	Dev A	IAbility: next kick impulse is multiplied by 2. Single use per rally. Emits 'ball:megabounce'.
ArenaVariationSystem.ts	STUB	Dev A	Picks random arena config before match. Randomises time-of-day, LUT, gravity modifier.

src/animation/

File	Status	Owner	Purpose
AnimationSystem.ts	STUB	Dev B	Holds AnimationGroup map per character. Plays clips by state. Phase 4: adds weight-based blending.
AnimationEvents.ts	STUB	Dev B	Registers frame callbacks on AnimationGroups. Fires 'kick:impact' at correct anim frame for timing.
IKController.ts	STUB	Dev B	Wraps BoneIKController. Optional — apply to foot bones for surface contact. Phase 5 only.

src/vfx/

File	Status	Owner	Purpose
VFXSystem.ts	STUB	Dev B	Implements IVFXSystem. Loads particle JSONs. playEffect(id, position) spawns correct emitter.
ParticleLibrary.ts	STUB	Dev B	Maps effect IDs to particle config JSON paths. Single source of truth for all particle effects.
ShaderLibrary.ts	STUB	Dev B	Loads NodeMaterial JSON shaders. Caches by ID. applyToMesh(id, mesh) for one-liner usage.
TrailManager.ts	STUB	Dev B	Creates and manages TrailMesh on the ball. Modulates width/color based on ball speed and active effects.

src/ui/

File	Status	Owner	Purpose
UIManager.ts	STUB	Dev C	Creates AdvancedDynamicTexture.CreateFullscreenUI. Owns all UI layers. show/hide by scene state.
LoadingScreen.ts	STUB	Dev C	Progress bar and logo shown during AssetManager loading phase. Hides on onReady.
MainMenu.ts	STUB	Dev C	Play, Settings, Character Select buttons. Animated 3D background scene.
CharacterSelect.ts	STUB	Dev C	Grid of character cards. 3D preview via RenderTargetTexture. Stats bars. Confirm selection.
ArenaSelect.ts	STUB	Dev C	Arena cards with preview image. Selects IArenaConfig to pass to ArenaVariationSystem.
HUD.ts	STUB	Dev C	In-match overlay: score, serve indicator, set counter, superpower cooldown bars.
PointAnnouncement.ts	STUB	Dev C	Animated full-screen overlay on point scored. Shows for 2s, fades out.
PostMatch.ts	STUB	Dev C	Winner display, stats table, Play Again / Menu buttons.
CooldownRing.ts	STUB	Dev C	Disc mesh below each player. Arc fills as cooldown recharges. Glows when power is ready.

src/audio/

File	Status	Owner	Purpose
AudioSystem.ts	STUB	Dev C	Implements IAudioSystem. Wraps BabylonJS Sound + Howler.js. play(id), playLoop(id), stop(id).
AudioLibrary.ts	STUB	Dev C	Maps audio IDs to file paths. Single source of truth. e.g. 'bounce_table' → 'assets/audio/bounce_table.ogg'.
MusicManager.ts	STUB	Dev C	Cross-fades between arena music tracks. Intensifies stem at match point. Stops on match end.

src/data/ (JSON — no TypeScript)

File	Status	Owner	Purpose
characters/flamingo.json	STUB	Dev C	CharacterConfig: id, displayName, modelPath, stats {speed,jump,power,spin}, superpower, limitation, animations map.
characters/human_athlete.json	STUB	Dev C	Second character config. Different stats and superpower from flamingo.
characters/character3.json	STUB	Dev C	Third character config. Placeholder until character is modelled.
arenas/cloud_arena.json	STUB	Dev C	IArenaConfig: skybox path, fogColor, gravityModifier, ambientParticles path, backgroundMusic path.
arenas/underground_rave.json	STUB	Dev C	Second arena config. Neon palette, pulse light trigger on beat.
arenas/space_station.json	STUB	Dev C	Third arena config. Near-zero gravity modifier, Earth skybox.

File	Status	Owner	Purpose
<code>abilities.json</code>	REAL	Dev C	All abilities: id, displayName, cooldown, duration, description. Drives UI display and SuperpowerSystem lookup.
<code>match-rules.json</code>	REAL	Dev A	Teqball rules: maxTouchesPerPlayer, netTouchFault, handTouchFault, setLength. Loaded by RuleEngine.

tests/

File	Status	Owner	Purpose
<code>ruleEngine.test.ts</code>	REAL	Dev A	Unit tests for RuleEngine pure functions. See Section 9.
<code>characterState.test.ts</code>	REAL	Dev A	Unit tests for character state machine transitions.
<code>superpower.test.ts</code>	REAL	Dev A	Unit tests for SuperpowerSystem cooldown and duration logic.
<code>eventBus.test.ts</code>	REAL	Dev A	Unit tests for EventBus: emit, subscribe, unsubscribe, no memory leak.
<code>matchFlow.test.ts</code>	STUB	Dev A	Integration test for full rally → point scored flow. Written in Phase 3.
<code>physicsSanity.test.ts</code>	STUB	Dev A	Reproducibility test: same initial conditions → same outcome. Written after physics locked.

assets/ (binary — all tracked by Git LFS)

File	Status	Owner	Purpose
<code>models/teqball_table.glb</code>	REAL	Dev B	Teqball table + net. Rigged for physics. Tagged meshes in metadata. Must exist before Phase 1.
<code>models/ball.glb</code>	REAL	Dev B	Ball mesh. Simple sphere. Multiple material slots for skin variants.
<code>models/characters/</code>	STUB	Dev C+B	One .glb per character. Contains mesh + rig + all animation clips merged.
<code>models/arenas/</code>	STUB	Dev C+B	One .glb per arena environment. Excludes table (loaded separately).
<code>textures/</code>	STUB	Dev B	All .ktx2 compressed textures. Never raw .png in this folder.
<code>textures/raw/</code>	STUB	Dev B	Source .png/.psd files. Not committed to Git (add to .gitignore). Local only.
<code>env/</code>	STUB	Dev B	HDR environment files in .env format. One per arena. Created via BabylonJS env tool.
<code>shaders/</code>	STUB	Dev B	NodeMaterial JSON exports from NME. One per visual effect (toon, outline, distort, lut).
<code>particles/</code>	STUB	Dev B	Particle system JSON exports from BabylonJS Particle Editor. One per VFX effect.
<code>audio/music/</code>	STUB	Dev C	.ogg music loops. One per arena. Encoded at 128–192kbps.
<code>audio/sfx/</code>	STUB	Dev C	.ogg SFX files: bounce_table, bounce_net, kick, score, fault, power_activate, power_deactivate.

docs/

File	Status	Owner	Purpose
<code>architecture.md</code>	REAL	Dev A	High-level system diagram (text-based). Describes how each system connects via EventBus.
<code>qa-checklist.md</code>	REAL	Dev C	Manual QA checklist. Filled in at end of Phase 3 and Phase 6.
<code>asset-guidelines.md</code>	REAL	Dev B	Size limits, compression commands, naming conventions for all asset types.
<code>onboarding.md</code>	REAL	Dev A	Guide for new contributors: clone, install, run, PR process, folder ownership.

Total files to stub on initialization: ~55 TypeScript files + JSON configs + documentation. Dev A creates all src/stubs in Phase 0 using a script or manually. Each stub exports an empty class with the correct name and implements the relevant interface. This ensures the TypeScript compiler validates imports from day 1.

Stub file template

```
// src/entities/Ball.ts – stub
import { IBallSystem } from '../core/interfaces';
import { Vector3 } from '@babylonjs/core/Maths/math.vector';
export class Ball implements IBallSystem {
  get velocity(): Vector3 { return Vector3.Zero(); } // TODO Phase 1
  applyKickImpulse(direction: Vector3, power: number): void { } // TODO Phase 1
  applyEffect(effectId: string): void { } // TODO Phase 4
  update(delta: number): void { } // TODO Phase 1
  dispose(): void { } // TODO Phase 1
}
```